



PDF Download
3208040.3208044.pdf
05 January 2026
Total Citations: 7
Total Downloads: 773

 Latest updates: <https://dl.acm.org/doi/10.1145/3208040.3208044>

RESEARCH-ARTICLE

Profiling distributed systems in lightweight virtualized environments with logs and resource metrics

AIDI PI, University of Colorado Boulder, Boulder, CO, United States

WEI CHEN, University of Colorado Boulder, Boulder, CO, United States

XIAOBO ZHOU, University of Colorado Boulder, Boulder, CO, United States

MIKE JI

Open Access Support provided by:

University of Colorado Boulder

Published: 11 June 2018

[Citation in BibTeX format](#)

HPDC '18: The 27th International Symposium on High-Performance Parallel and Distributed Computing
June 11 - 15, 2018
Arizona, Tempe

Conference Sponsors:
SIGARCH

Profiling Distributed Systems in Lightweight Virtualized Environments with Logs and Resource Metrics

Aidi Pi, Wei Chen
Department of Computer Science
University of Colorado
Colorado Springs, CO, USA

Xiaobo Zhou
Department of Computer Science
University of Colorado
Colorado Springs, CO, USA

Mike Ji
Datacenter Solutions
Hillstone Networks Inc.
Santa Clara, CA, USA

ABSTRACT

Understanding and troubleshooting distributed systems in the cloud is considered a very difficult problem because the execution of a single user request is distributed to multiple machines. Further, the multi-tenancy nature of cloud environments further introduces interference that causes performance issues. Most existing troubleshooting tools either focus on log analysis or intrusive tracing methods, leaving resource usage monitoring unexplored.

We propose and implement LRTrace, a non-intrusive tracing and feedback control tool for distributed applications in lightweight virtualized environments. LRTrace profiles both log messages and actual resource consumptions of an application at runtime in a fine-grained manner, which is made possible by lightweight container-based virtualization. By correlating these two kinds of information, LRTrace provides users the ability to build the relationship between changes in resource consumption and application events. Furthermore, LRTrace allows users to define and implement their own feedback control plug-ins to manage the cluster in a semi-automatic manner. In system evaluation, we run Spark and MapReduce applications in a multi-tenant cluster and show that LRTrace can diagnose performance issues caused by either interference or bugs, or both. It also helps users to understand the workflows of data-parallel applications.

KEYWORDS

Troubleshooting, Lightweight virtualization, Logs, Resource metrics, Data-parallel applications

ACM Reference Format:

Aidi Pi, Wei Chen, Xiaobo Zhou, and Mike Ji. 2018. Profiling Distributed Systems in Lightweight Virtualized Environments with Logs and Resource Metrics. In *HPDC '18: The 27th International Symposium on High-Performance Parallel and Distributed Computing*, June 11–15, 2018, Tempe, AZ, USA. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3208040.3208044>

1 INTRODUCTION

Understanding and troubleshooting distributed systems in cloud environments is critical but very difficult [24, 25, 33]. In a distributed

system, a single user request can be divided into pieces and processed in components on different nodes in the cluster. One anomaly may slow down or even fail the entire application. Due to the distributed nature, the root cause of a problem is often diverse: software bugs, interference, misconfigurations, data skews, etc.

Extensive studies focus on anomaly and bug detection. One major method is plain log reconstruction and analysis [18, 33]. It needs to consider the tradeoff between the effectiveness and overhead. In other words, extracting more logs improves the effectiveness but also introduces more overhead. Another popular method is intrusive tracing [15, 24]. This approach ensures the accuracy since the information is extracted from inside the software. The essential problems, however, are that 1) users need to have a deep understanding of the targeted systems, and 2) modifications to one version of the source code will be invalid after the targeted system is updated.

Besides the methods above, few studies utilize resource metrics to detect anomalies and bugs [19]. One reason is that resource metrics are usually collected at the granularity of per machine. As multiple processes usually run on one machine simultaneously, the resource metrics are inaccurate for troubleshooting a specific process. For each process, it is straightforward to access the CPU and memory usages. But I/O metrics of a process are hard to obtain.

Lightweight virtualization techniques, such as Docker [1] and LXC [3], have been gaining increasing interests in both academia and industry due to their higher performance, lower overhead as well as competitive isolation compared to traditional virtualization such as KVM [23] and VMWare. An additional opportunity provided by lightweight virtualization is that it allows the access to more fine-grained resource metrics. Furthermore, each lightweight virtual container has a unique identifier. By leveraging the identifiers, we can distinguish the resource usages of different applications.

In this paper, we propose and develop LRTrace, a distributed troubleshooting and feedback control tool. LRTrace correlates both log and fine-grained resource metrics that are made possible by lightweight virtualization. LRTrace focuses on profiling data analytics frameworks such as Spark [31] and MapReduce [20]. It helps users to reconstruct the workflows, understand the frameworks, find the anomalies and locate their root causes. Furthermore, LRTrace has a pluggable user-defined feedback control component. Based on the runtime information of applications that is collected by LRTrace, users can write plug-ins to control the behavior of applications thus manage the cluster in a semi-automatic way.

Extracting and reconstructing logs of distributed systems are, however, non-trivial due to the following reasons:

- Profiling within one log message: a log message usually contains multiple types of information, such as timestamps, object identifiers and the amount of processed data. Also,

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

HPDC '18, June 11–15, 2018, Tempe, AZ, USA

© 2018 Association for Computing Machinery.

ACM ISBN 978-1-4503-5785-2/18/06...\$15.00

<https://doi.org/10.1145/3208040.3208044>

the format of log messages varies. To parse one log message, the tracing tool needs to identify different fields and the corresponding meanings within one log message.

- Profiling across log messages: the same object appears in different log messages indicating different events. For example, an object may be recorded in three log messages. The first one suggests the start of the object, the second one records the amount of data processed by the object, and the third one is the end mark of the object. To reconstruct the workflow, we need to identify the same object that appears in different log messages.

To address the challenges, we propose *keyed message*, a uniform structure of log messages. A keyed message is a key-value like tuple but contains more fields. Operations on keyed messages, such as Groupby, Count and Sum, can be easily done in order to reconstruct the workflow of an application. By defining simple rules, users can transform intended log messages to keyed messages. In practice, we also use keyed messages to store resource metric information. This design facilitates both user requests and backend storage.

LRTrace correlates log and resource metrics by matching the intrinsic identifiers of an application. The relationship between the two kinds of information is crucial to understand the targeted systems. Furthermore, an anomaly usually happens when a ‘mismatch’ of log messages and resource consumption is found. The feedback control component is used to execute user-defined plug-ins, which is especially useful to handle some unusual cases that are crucial to cluster performance but are not handled by the cluster scheduling framework, such as putting a bottlenecked node in the blacklist so that no incoming task should be assigned to the node.

In our design, we develop two major components: Tracing Worker and Tracing Master. Tracing Worker runs on every worker machine. It is responsible for collecting both logs and resource metrics, and sending them to a collection component such as Kafka [9]. Tracing Master pulls the information from the collection component. After been transformed to keyed message, the information is presented to users and sent to the time series database such as OpenTSDB [5] and Graphite [2]. Tracing Master is also responsible to execute the user-defined plug-ins that are used to perform feedback control based the runtime information.

Experimental results show that LRTrace is powerful in reconstructing the workflows of applications such as Spark and MapReduce. We evaluate LRTrace by profiling interfered applications and buggy applications. Using LRTrace, we find a bug (SPARK-19371 [6]) that is recently reported by other users. Furthermore, we find a new bug (YARN-6976 [8]) in Apache Yarn. Experiments also show that LRTrace helps to diagnose anomalies that share similar symptoms but are actually caused by different reasons. Furthermore, we implement two plug-ins to perform feedback control. One reduces the application execution time and improves throughput of the cluster. The other retries stuck or failed applications to make the applications successfully finish.

Our contribution lies in the following aspects. We build LRTrace, a distributed troubleshooting system that takes advantage of both logs and resource metrics by leveraging the keyed message. LRTrace is powerful in reconstructing the workflows as well as identifying bugs caused by various reasons while introducing low overhead to

the targeted systems. By using user-defined feedback plug-ins, LRTrace is able to perform feedback control, which provides flexibility for administrators to manage the cluster under different situations.

The rest of the paper is organized as follows. In section 2, we give a motivating example. Section 3 describes the design of keyed message. Section 4 presents the overview of LRTrace. Section 5 evaluates LRTrace. Sections 6 discusses the limitations of LRTrace. We present related work in Section 7 and conclude in Section 8.

2 MOTIVATING EXAMPLE

We use a motivating example to demonstrate what information LRTrace can provide to users and how the information is presented. We run a Spark KMeans job with the large data set generated by HiBench [22] on a 9-node cluster that is managed by Yarn [27] and use Docker as the lightweight virtual container. We demonstrate how a user monitors this application using a traditional method compared to using LRTrace.

Traditional approach. We refer the traditional approach to debugging with tools only provided by the application. Most applications provide two kinds of tools: 1) the raw logs, and 2) the web server provided by the App Master. Logs record various objects and events in a chronological manner and are quickly generated. Thus, log messages regarding to an object may be scattered across the whole log file. Another problem is that logs in a distributed system are generated in different files. A user may spend a long time in finding a specific object in logs even after the application finishes. A study [33] has shown that using raw logs for anomaly analysis is a common but an inefficient approach.

The web server provides an easy access to the workflows of the application. However, the information recorded in the web server is determined by the framework developers. Some applications record less information in the web server than in the logs. For example, the web server of Hadoop MapReduce only records general information about tasks such as the location, progress and state. Detailed information such as shuffle or spill events cannot be obtained from the web server. Some other applications such as Spark record different information in the web server from logs. In general, the web server can be used as a complementary tool for debugging. However, both approaches lack flexibility of applying operations on the objects.

In this example, suppose a user wants to get the information about tasks in each Spark executor. The user might not read the raw log files since this approach is too time consuming. Instead, the Spark web server provides information about each task such as its location, its start/end time and its input size, which only presents the information of individual tasks but is insufficient for an overview on all tasks.

LRTrace. By leveraging the keyed message, LRTrace provides concise information that describes the workflow at runtime. We now show an intuitive view of how keyed messages work, and leave the details of its design to Section 3. Figure 1 shows a small fraction of information that can be provided by LRTrace. Since the application is administrated by Yarn and uses Docker containers, we can access both logs and fine-grained resource metrics of each container. A Spark application is divided into multiple stages. In each stage, each container executes several tasks. For the conciseness of demonstration, we only present information about three

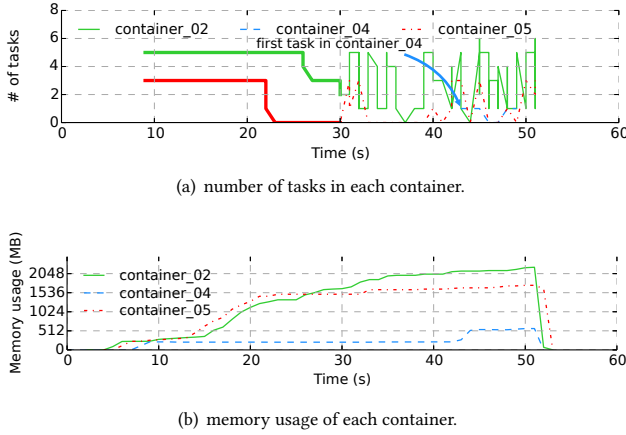


Figure 1: Number of tasks and memory usage of a HiBench KMeans application.

containers of the application. Figure 1(a) describes the number of tasks concurrently running in each container, which is obtained by the following request:

```
key: task
aggregator: count
groupBy: container, stage
```

This request first sets the key as *task*. The count aggregator counts the number of active tasks during each profiling interval. The results are grouped by the container IDs and stage IDs that are identifiers attached to keyed messages. In Figure 1, we highlight tasks in stage_0 with thick lines to distinguish them from those in other stages. Between 22s and 30s, container_02 is processing tasks in stage_0 while container_05 (and several other containers which are not shown in the figure) has finished the tasks in the current stage and becomes idle. At this point, a user can clearly identify container_02 as a straggler in stage_0. It is also obvious that container_05 gets assigned fewer tasks than container_02 does throughout the entire lifespan of the application.

LRTrace also provides the resource consumption of a container. Figure 1(b) shows the memory usages of the three containers, obtained by the following request:

```
key: memory
groupBy: container
```

From Figure 1(b) a user can find that three containers start at around the same moment. However, container_04 gets assigned its first task right before the end of the whole application. The idle container occupies more than 200 MB of memory resource for a long time from its start. Both cases of task imbalance are caused by a bug in the Spark scheduler. We describe how LRTrace locates the bug in Section 5.

LRTrace provides flexibility to reconstruct the workflow from logs. Reconsider the request that generated results in Figure 1(a). The request returns the number of tasks per container and stage. If a user wants to inspect the total number of running tasks in the whole cluster, the user only needs to remove “container” from the aggregator field. We also use requests of the same format to obtain resources metrics, shown in Figure 1(b).

Field	Description
key	the key assigned to a message
identifiers	to identify the object in the message
value	a numeric variable to store the value in the message
type	type of the message, <i>instant</i> or <i>period</i>
is-finish	whether the message indicates object lifespan end
timestamp	the time when the message is written

Table 1: Description of fields in a keyed message.

Summary Compared to traditional approaches, LRTrace provides a concise view of the events recorded in the logs. By applying operators, in this case sum on tasks of each container, users can reconstruct the workflow from a statistical aspect. Also, correlating resource metrics gives users an insight on how an event affects the resource consumption.

3 KEYED MESSAGE

Logs generated by distributed systems record rich information including system initialization, lifespans of objects, current states of the system, error messages, etc. However, even experienced programmers have to spend a long time in locating the root cause of anomalies when using raw logs [33]. Our observation indicates that using raw logs is not an efficient method for manual analysis:

- Information in log messages is rich but also miscellaneous. Since logs record all kinds of information, a log message of interest is usually surrounded by other irrelevant ones. Manually locating such a log messages is time-consuming and inefficient.
- Workflows recorded in logs are not structured. Popular logging tools, e.g. log4j [10] and slf4j [11], output log messages in a time-ordered manner. Messages recording events of the same object are usually separated by other messages. For example, Yarn ResourceManager outputs log messages when the application state transitions from one to another. If multiple applications run simultaneously, the state transition messages of different applications may be intertwined.

To address the problems, we propose *keyed message*, which can be transformed from a raw log message by applying simple rules. Keyed messages provide the ease to aggregate messages with the same key, as well as the flexibility for user to define their own keys.

3.1 Log Transformation

We use regular expressions to extract intended log messages. This approach seems to be *ad hoc*, but we use it for two reasons: 1) Though field identification is a solved problem using tools such as Logstash [4], additional information cannot be attached to a log message. In our case, we attach a key and identifiers to a log message. 2) Log messages concerning the workflow repeatedly appear with only few changing fields, despite the fact that the total number of log messages is large. For example, in Figure 2 messages at line 1 and line 3 only differ in the task ID. In evaluation, we show that a limited number of regular expressions are enough to

```

1 Got assigned task 39
2 Running task 0.0 in stage 3.0 (TID 39)
3 Got assigned task 41
4 Running task 1.0 in stage 3.0 (TID 41)
5 Task 39 force spilling in-memory map to disk and it
  will release 159.6 MB memory
6 Task 41 force spilling in-memory map to disk and it
  will release 180.0 MB memory
7 Finished task 0.0 in stage 3.0 (TID 39)
8 Finished task 1.0 in stage 3.0 (TID 41)

```

Figure 2: A snippet of simplified log output by a Spark application. Extracted fields are shown in different colors.

Line	Key	Id	Value	Type	is-finish
1	task	task 39	-	period	F
2	task	task 39	-	period	F
3	task	task 41	-	period	F
4	task	task 41	-	period	F
5	spill	task 39	159.6 MB	instant	-
	task	task 39	-	period	-
6	spill	task 41	180.0 MB	instant	-
	task	task 41	-	period	-
7	task	task 39	-	period	T
8	task	task 41	-	period	T

Table 2: Keyed messages corresponding to logs in Figure 2.

capture the whole workflow of an application. To be specific, we use 12 rules, 4 rules and 5 rules to extract the workflow of Spark, MapReduce and Yarn, respectively.

A keyed message consists of five fields in order to describe a raw log message. Table 1 lists the fields of a keyed message. *Key* represents the high-level objects or events in a log message. *Identifiers* are used to uniquely identify the object or event in a message. *Value* stores a numeric value recorded in a log message if applicable. *Type* shows that the object in a log message is either an instant event or a period object. Field *is-finish* is a flag indicating whether the message is the end mark of a period object and is only applicable to messages of *period* type.

We illustrate how to transform raw log messages to keyed messages by using a snippet of log messages output by a Spark application in Figure 2. Messages 1 and 3 indicate that the Spark executor gets the tasks (task 39 and 41). During the processing of tasks, each task spills a certain amount of data to disk, which is recorded in messages 5 and 6. Messages 7 and 8 suggest the tasks are finished. Note that log messages 5 and 6 are both transformed to two keyed messages, respectively. One indicates the execution of the task, and the other indicates the spill event. We simplify the logs by omitting timestamps and classes that generate the messages. In practice, we indeed extract timestamps recorded in log messages.

Users can use a configuration file in *.xml or *.json format to define the rules extracting log messages. Then, the Tracing Workers parse the configuration file and transformed the specified log messages to keyed messages. In our implementation, we use *.xml as the format of the configuration file. The following snippet of rules illustrates how we extract log message 1 in figure 2.

```

<rule>
  <regex>Got assigned (?&lt; taskId &gt; task [0-9]+)<\regex>
  <key-group>
    <key>task<\key>
    <identifier>taskId<\identifier>
    <is-period>true<\is-period>
    <is-finish>false<\is-finish>
  <\key-group>
</rule>

```

To summarize, we transform the eight messages in Figure 2 to keyed messages and show them in Table 2. Taking advantage of the design of keyed messages, operations such as Groupby, Count and Sum can be easily applied for data analytical jobs. Furthermore, users can perform feedback control at runtime based on the keyed messages. For the ease of use, we provide users with configuration files for Spark and MapReduce applications. Besides, users can alter the existing rules or define new rules to extract intended log messages for other systems.

3.2 Resource Metrics Storage

LRTrace uses keyed messages to store resource metrics. The advantage of using keyed messages is that users can request all information in a uniform format.

A keyed message stores resource consumption information. In LRTrace, a profile of resource metrics contains four fields: the metric name, the value of usage, its container ID and the profiling time, which are the counterparts of *key*, *value*, *identifier* and *timestamp*, respectively, in a corresponding keyed message with *type* of period. Field *is-finish* is set to be false except for the last metric of a container. Thus, users can regard a resource metric as a special case of a period event whose lifespan equals to the corresponding container.

4 LRTRACE DESIGN

We choose Yarn as the cluster resource management framework since it is widely used in both academia and industry. The basic idea can be extended to other cluster resource managers such as Mesos [21]. In this section, we first review the architecture of Yarn. We then describe the design of LRTrace.

4.1 Yarn

Apache Hadoop Yarn [27] is a popular resource management framework that can serve for applications like MapReduce and Spark. Resources are packed in containers. For example, a Yarn container can have {2 cores, 4 GB RAM} as the resources. An application requests containers from Yarn and launches jobs in them. Traditionally, containers run on top of bare operating systems. Resource consumption of a container is constrained in a logical manner. An alternative choice is to launch a container in a lightweight container-based virtualized environment, which provides more flexibility for resource allocation and accuracy for resource monitoring. To avoid ambiguity, we refer ‘container’ to the container in Yarn, and ‘LWV container’ to the lightweight virtualized container unless otherwise specified in the rest of the paper.

In a cluster, containers are assigned with unique identifiers. In the extreme case, two containers may share the same identifier when the date on the machine is changed, which rarely happens in a real production cluster. Our work does not consider this extreme

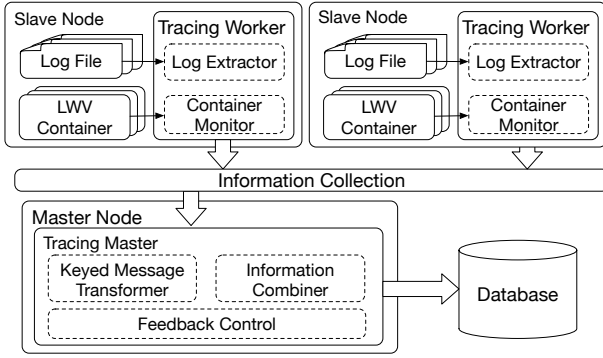


Figure 3: LRTrace architecture.

situation. Leveraging the uniqueness of container IDs, we correlate log messages and resource metrics belonging to the same container.

4.2 LRTrace Architecture

LRTrace is a distributed framework designed to collect and correlate both log messages and resource metrics at runtime. We show the high-level architecture of LRTrace in Figure 3. The information collection component and database are considered external to LRTrace. The Tracing Worker runs on each node in the cluster. It reads information from LWV container API files and the log files. After being pre-processed by the Tracing Worker, the information is sent to the information collection component and pulled by the Tracing Master. The Tracing Master is responsible for transforming raw log messages to keyed messages, correlating keyed messages with resource metrics, and sending all the information to a database for further analyses.

4.3 Tracing Worker

Log collection. There are two kinds of logs: Yarn logs generated by ResourceManager or NodeManager, and application logs generated by applications running in Yarn containers. We assume that all the intended log messages follow the format below which is supported by most of the popular logging tools:

```
timestamp: log contents
```

The Tracing Worker collects both kinds of raw logs and sends them to the information collection component. In order to provide container IDs and application IDs for the Tracing Master to construct a keyed message, the Tracing Worker also attaches these two identifiers to a raw log message. For Yarn logs, these identifiers can be extracted from the log messages. However, application logs are written by application developers and only contain objects that are internal to the application. Since applications adopt Yarn as the resource manager, the directory path of an application log file contains the information about the application ID and the container ID. For example, the path of a Yarn log file may look like `$HADOOP_HOME/application_01/container_01_01`. The Tracing Worker extracts these identifiers by parsing the absolute paths of application log files.

Resource metrics. Metrics are collected via APIs provided by LWV containers. The Tracing Worker monitors four major resources:

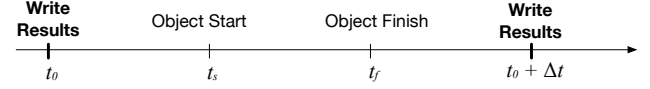


Figure 4: Illustration of how a short object can be missed.

CPU, memory, disk I/O and network I/O. In order to distinguish resource metrics of one LWV container from those of another, the Tracing Worker associates resource metrics with the corresponding Yarn container ID and sends them to the information collection component.

The sampling frequency of monitoring is a trade-off between overhead and accuracy. Higher frequency leads to more overhead of CPU, memory and storage. For long jobs, a low frequency is enough to capture the behavior of resource consumption. However, short jobs lasting only tens of seconds require a higher frequency to guarantee accuracy. In experiments, we set the frequency to 1Hz for long jobs and 5Hz for short jobs.

4.4 Tracing Master

The Tracing Master first pulls information (raw logs and resource metrics) from the information collection component. Then, it transforms raw log messages to keyed messages. It periodically writes the processed information to users or the database. Each wave of outputs includes the resource metrics, the living period events and the newly received instant events during the last monitoring interval. The Tracing Master is also responsible for executing user-defined feedback control plug-ins.

Keyed message construction. For instant events, the transformation is straightforward. The Tracing Master simply extracts all the required fields and puts them in the output buffer. For period objects, the Tracing Master maintains a living object set. Objects in the set are distinguished by their keys and identifiers. The Tracing Master adds a period object to the set upon first receiving it, and keeps the object in the set until receiving a corresponding message that has a true *is-finish* flag.

Since transforming messages and writing results are asynchronous, chances are that two messages containing the same period object are received within one writing interval. This is common for an object with a short lifespan. Figure 4 illustrates the scenario. There are two consecutive writing operations at t_0 and $t_0 + \Delta t$, respectively. In between them, an object starts at t_s and finishes at t_f . In this case, the object is removed before the second writing operation, which causes data lost. To avoid this situation, the Tracing Master keeps a finished object buffer. When a period message with a true *is-finish* flag arrives, the Tracing Master adds the removed object to the finished object buffer. Each writing includes objects in both the living object set and the finished object buffer. After each writing, the Tracing Master empties the finished object buffer.

Log and resource metrics matching. As discussed in Section 2, every keyed message is attached with an application ID and a container ID, so are the resource metrics. A matching is done by associating keyed messages and resource metrics that share the same identifier. Since the timestamp granularities of keyed messages and resource metrics may vary, we do not use timestamps

when matching the two kinds of information. Instead, the Tracing Master presents the information on two timelines respectively in a chronological manner, one for events in logs and the other for resource metrics.

Feedback control. The information collected by Tracing Worker reflects the current status of the cluster, which includes the state and resource consumption of each application. This information can be used to manage the cluster immediately, such as adjustment of the resources consumed by applications and trivial error handling.

To assist semi-automatic cluster management, LRTrace provides a programming interface for users to define their own feedback control plug-ins. For the ease of use, LRTrace does not provide users with the raw data through the interface. Instead, Tracing Master arranges the data collected by Tracing Worker into time-sliding windows. The window size can be specified by the users. In each window, the information is presented in the format of the keyed message, and is grouped by the application ID and container ID. The interface contains an `action(data window)` function that will be called by Tracing Master periodically and shall be implemented by users. After obtaining the data, users can design their feedback logics to manage the cluster automatically. Usually, users follow three steps when implementing the `action(data window)` function. Firstly, users get cluster status data extracted from keyed messages via the interface. Secondly, users update their own local variables based on the cluster data. These variables, for example, can be the usage of resources during the last few windows or counters indicating timeout. Finally, users execute cluster management tasks if their local variables meet certain conditions.

Data Query. We design the keyed message to be compatible with the query language used by the backend database, OpenTSDB in our implementation. The query language supports various operations on the keyed message, to name a few, Count, Groupby, Average, downsampling, changing rate calculation, etc. These operations facilitate analysis on the traced data. For example, changing rate calculation can be used to calculate the network/disk usage rate according to the cumulative usage. The Count operation gets the number of concurrently running objects at runtime.

5 SYSTEM EVALUATION

In this section, we evaluate LRTrace from the following three aspects. 1) We use an example to illustrate how LRTrace reconstructs the workflow (§5.2). We also show some interesting findings in this case. 2) We use several case studies to show how LRTrace can help users to troubleshoot the systems (§5.3 & §5.4) and manage the applications (§5.5). 3) We further show the performance overhead introduced by LRTrace (§5.6).

5.1 Testbed Setup

We conduct the experiments on a 9-node cluster (1 master and 8 slaves). Each machine has an Intel(R) Core(TM) i7-2600 CPU, 8GB of memory and a 512GB of HDD with 7200 rpm. The cluster is connected by 1Gbps Ethernet. Experiments run on the Ubuntu 16.04 LTS 64-bit operating system. We use kafka-0.10.2.1 as the information collection component and OpenTSDB-2.3.0 as the time series database. We use the GUI web server provided by OpenTSDB for data visualization and analysis. For Spark applications, we run the

original Spark-2.1.0 on Hadoop-2.7.3. We use the sequenceiq/hadoop-docker-2.4.0 as the LWV container. Experiment data are generated either from HiBench-6.0 [22] benchmark or TPC-H [7] benchmark. Detailed information about the data sizes is given in each experiment, respectively.

5.2 Workflow Reconstruction and Analysis

Spark. In this experiment, we run a Spark Pagerank workload to illustrate how LRTrace reconstructs the Spark workflow. The Spark Pagerank workload is configured to perform three iterations on 500MB input data. For log extraction, we define only 12 rules, which is enough to capture the whole workflow. Table 3 summarizes these rules. This application is configured with eight executors. For conciseness, we only demonstrate the behaviors of three representative containers but omit the rest of them since they have similar behaviors to one of the three.

We show the states of the application attempt and two containers in Figure 5. In this figure, users can see the states of each component at a given time and the length of each state. The RUNNING state of a container is further divided into two sub-states: initialization and execution. Once a container enters the RUNNING state, it starts the internal initialization before it actually receives tasks. The internal state is transparent to Yarn and recorded in the application logs. LRTrace captures the internal states by assigning the same key to related log messages from either the application or Yarn.

Figure 6 shows resource metrics and the related log events. Figure 6(a) shows the CPU usages of two containers. Container_03 and container_06 finished the inner initialization at 10s and 12s respectively. Then, the tasks pre-process the data until 74s. From 74s to 93s, there are three CPU usage peaks indicating the three iterations. Finally, from 93s to 96s, the result is written to the disk.

Analysis on memory behavior Findings get more interesting when we inspect the memory usage and its related events. Figure 6(b) shows that the memory usages of container_03 and container_04 drop immediately at certain moments (container_03 at 49s and container_04 at 58s and 69s). Only container_03 has a spilling operation indicated by the dashed vertical green line. Also, the decrease in memory of container_03 happens a few seconds later than the spilling event. We infer that swapping and JVM garbage collection (GC) are two possible reasons for a decrease in memory usage. We check both swap memory usage of the LWV containers and the GC log of the JVM. The swap memory usage stays under 30MB during the entire execution, which apparently does not cause the drop in memory. On the other hand, the GC log indicates that a full garbage collection occurs every time when the memory drops (a full garbage collection does not always cause a drop in memory usage). This explains the delay between the spilling event and the memory drop in container_03. A spilling only copies data to the disk. Later, a full garbage collection releases the memory. Table 4 summarizes the memory behaviors of container_03 and container_04. The amount of decreased memory is always less than the memory released by GC since tasks keep generating data during the execution.

Figure 6(c) shows the cumulative network usage of containers and shuffle events. A shuffle operation incurs a network I/O operation thus increases cumulative network usage. The key finding here

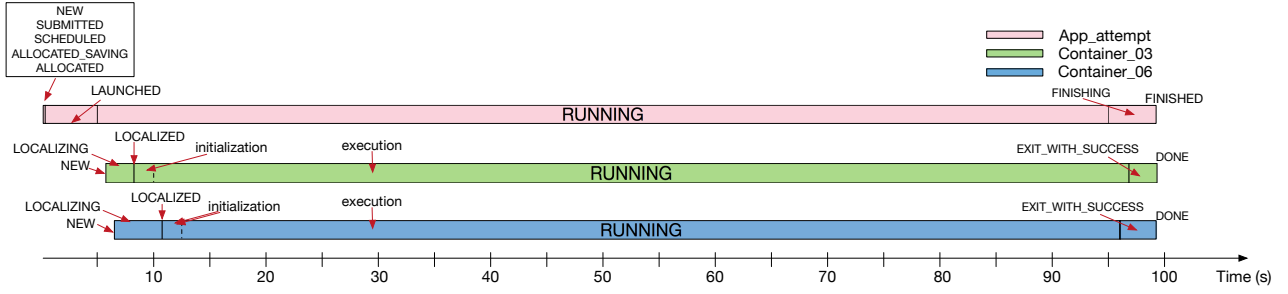


Figure 5: State machines of application attempt and two representative containers. The names of the states are written in capital letters. Each of short states is labeled with an arrow pointing to it.

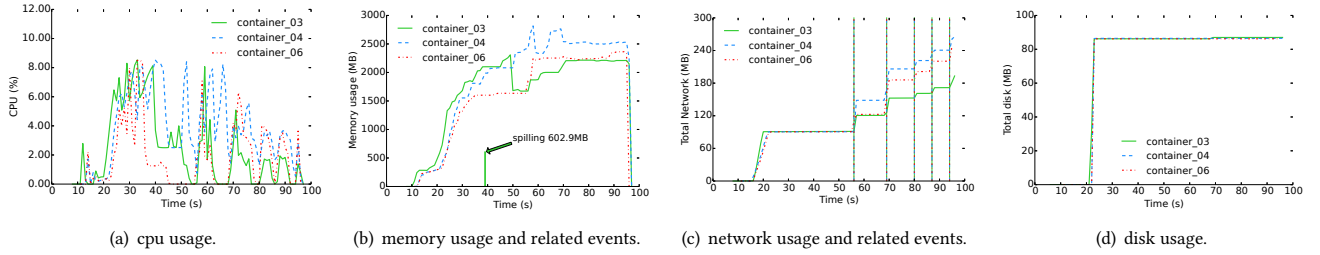


Figure 6: Resource metrics and events in three representative containers. The dashed vertical line in (b) represents a spilling event in container_03. The green and the dashed vertical blue lines in (c) represent shuffling operations (*period* events) in container_03 and container_06 respectively. (c) and (d) show cumulative I/O usage instead of instant I/O rate.

Object / Event	# of rules	Description
task	3	All three rules extract the corresponding stage ID, one for the start of a task, two for the end of a task.
spill	2	Both extract the processed data, one for task spilling events, the other for force spilling events.
shuffle	2	One for the start of a shuffle event, the other for the end.
container state	2	One for the start of a container, the other for the rest of state transitions.
application state	2	One for the start of an application, the other for the rest of state transitions.

Table 3: Summary of rules for extracting the workflow of a Spark application.

is that different containers in the Spark application always start shuffling at the same time, in this case at 56s, 69s, 80s, 87s and 94s, which are also the boundaries between stages. This indicates that the Spark application uses a synchronizing mechanism between stages so that it starts a new stage only after all the tasks in the previous stage are finished.

MapReduce. LRTrace is also capable to reconstruct the workflow of MapReduce applications. Slightly different from Spark, a task in MapReduce monopolizes one container. In this experiment, we run a Hadoop MapReduce Wordcount application on 3GB input data.

We demonstrate the workflows by showing one map task and one reduce task, respectively.

The map task mainly outputs two kinds of events concerning the workflow in its log: spill and merge. Figure 7(a) shows the workflow of the map task. After initialization, the map task quickly starts five consecutive spill operations. Note that compared to the first three spill, the fourth spill processes a similar amount of data but spends a shorter time. Then, the map task finishes 12 consecutive merge operation in a short time. Each merge processes about 6KB data.

The reduce task also outputs two kinds of events: fetcher and merge, shown in Figure 7(b). The reduce task first launches three fetchers to fetch the intermediate results generated by map tasks. Note that fetcher#2 starts later than the other two do. After the fetchers are finished, the reduce task starts to process the data, which is not recorded in the logs. Finally, the reduce task executes two merge operations, each on 30KB data.

We also find an abnormal map task. It starts much later than all the other map tasks and keeps alive for 27s even after the application is finished. In our further experiments, we find that a bug causes this anomaly. We explain how we find the bug in details next.

5.3 Bug Diagnosis

We describe how we find two bugs in the Spark-on-Yarn stack step by step. The first bug exists in the Spark scheduler (SPARK-19371 [6]) recently reported by other users. The other bug exists in Yarn ResourceManager (YARN-6976 [8]) which is reported by us.

Bug1: Uneven task assignment. The Spark-on-Yarn stack uses a two-level scheduler model. Spark ApplicationMaster first requests

Container	GC start	GC delay	Decreased memory	GC memory
container_03	49 th s	10s	658.7 MB	1083.9MB
container_04	58 th s	-	506.5 MB	1077.3 MB
container_04	69 th s	-	236.1 MB	1043.6 MB

Table 4: Summary of memory behavior. GC start is the moment when GC starts. GC delay is the time between a spilling event and a later full GC.

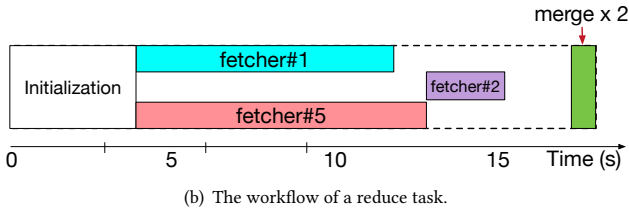
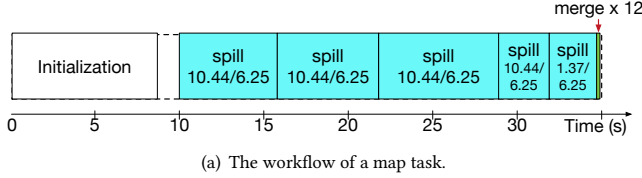


Figure 7: Workflows of a map task and a reduce task. The values below spill represent how many MB of keys and values are processed by the spill event, e.g. 10.44/6.25 stands for 10.44MB of keys and 6.25MB of values.

containers from the Yarn scheduler (level-1). After receiving containers, the Spark scheduler assigns tasks to them (level-2). The bug identified causes an uneven distributed number of tasks in different containers. To start with, we run a Spark TPC-H job (Query 08) with a MapReduce randomwriter as the interference. The Spark TPC-H job queries on a 30GB date set stored on HDFS. The MapReduce randomwriter writes 10GB data on each node of the cluster. We begin the debugging since we notice that some containers have considerably higher memory consumption than the others do, shown in Figure 8(a). We do not show container_01 since it runs ApplicationMaster that has stable memory consumption. Container_02, 04, 07 and 09 consume more than 1.4GB memory while container_03, 05, 06 and 08 only consume around 500MB memory. An uneven memory consumption indicates an uneven memory of data generated by tasks. This leads us to inspect the number of tasks assigned to each container. The following request gets the total number of running tasks during each 5-second downsampler interval:

```
key: task
groupBy: container
downsampler: {
  interval: 5s
  aggregator: count }
```

Figure 8(d) shows that containers which have higher memory consumption also receive more tasks than the other containers do. From the 2nd to the 10th interval, container_04 consecutively runs

more than 10 tasks in each interval while container_06 does not receive its first task until the 9th interval. This suggests that the Spark scheduler prefers to assign tasks to a container that previously receives tasks, which is caused by data locality across Spark stages. In other words, tasks are likely to be assigned to nodes that store the required data generated in the previous stage. However, data locality does not explain where in the first stage the scheduler assigns tasks. We conduct the experiments several times. The results suggest that the overloaded nodes in the first stage are random. We suspect that the scheduler prefers to assign tasks to containers which finish the initialization process early. Thus, we inspect the delay that a container enters the RUNNING state and the internal execution state, respectively, using the following request:

```
key: state
groupBy: container
```

The results in Figure 8(c) confirm our presumption. The scheduler assigns tasks to the four containers that finish initialization early. Note that although container_08 enters the RUNNING state early, it spends a long time on initialization, missing the chance to receive tasks. It is enough for a common user to stop here and claim that the Spark scheduler unevenly assigns tasks to different containers. But we further find that the unbalance is also prevalent in Spark applications even without interference.

We conduct experiments using other Spark workloads from Hi-Bench and TPC-H benchmarks with and without interference to check whether an unbalanced task assignment happens. These workloads include 30GB Wordcount, 30GB TPC-H Query 08, 30GB TPC-H Query12 and 10GB KMeans. We run each workload three times and use the average values as the results, shown in Figure 8(b). The length of the bar shows the difference between the maximum and minimum memory usage among containers. Note that we divide the KMeans job into two parts – part 1 includes the tasks before iterations and part 2 includes the tasks during iterations. The results show that an unbalanced memory distribution also exists in applications (or phases) Wordcount, TPC-H, and KMeans part 1 even without interference. These applications share a common feature that most of the tasks finish within one second.

This bug lies in the Spark scheduler, which causes unbalanced container memory consumption and introduces memory usage inefficiency. The root cause is that the Spark scheduler cannot make appropriate assignment decisions for sub-second tasks. Interference aggravates the unbalance since it causes late start of containers. Any container, even an idle one, still consumes a fixed amount of overhead memory. This memory is indispensable to launch the JVM. For the containers running Spark applications, the overhead memory is around 250MB. Once a container receives tasks, its memory usage increases due to the data generated by the tasks. We refer the increased memory usage on top of the overhead to effective memory. In another word, the actual memory usage of a container is the sum of the overhead memory and effective memory. Therefore, a higher memory consumption of a container means a larger amount of effective memory, which indicates the container is more effectively used. However, the effective memory of a container may also become a wastage of resources later. Consider the situation that a container first receives tasks then becomes idle. This container causes more wastage since the early tasks usually generate data, which increases the memory consumption of the container if it

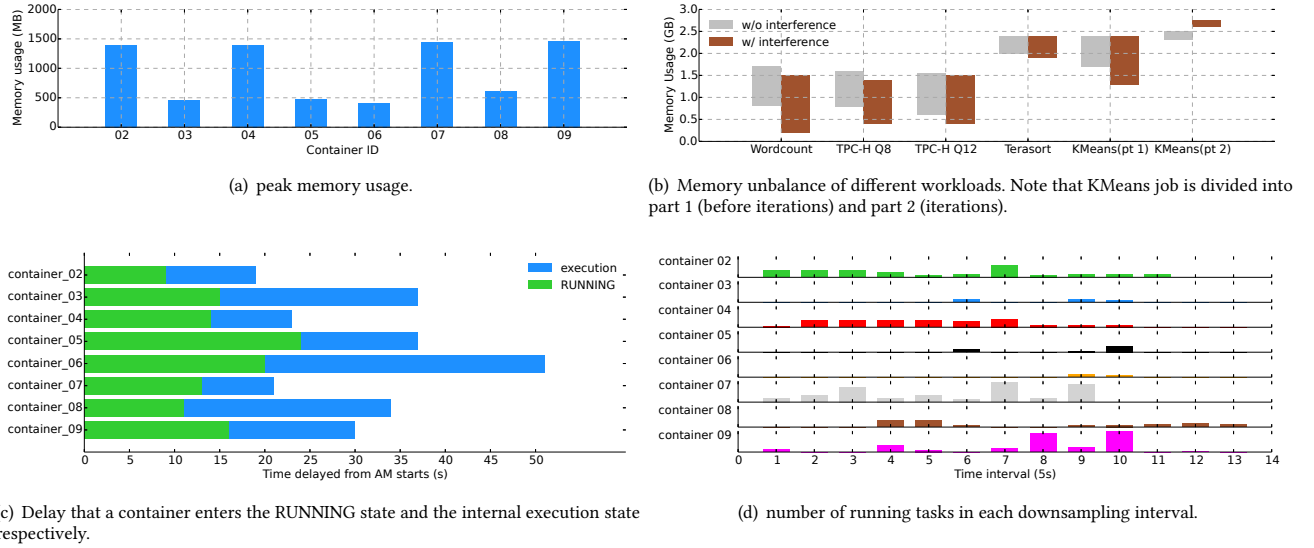


Figure 8: Related performance metrics when debugging SPARK-19371 and YARN-6976.

later becomes idle. An ideal scheduler should keep every container busy during the execution of an application so as to fully utilize the resources and also reduce the makespan.

Bug2: Zombie containers (new bug). Yarn ResourceManager is not aware of zombie containers. These containers stay alive for a long time after the application is finished, which wastes cluster resources. Running a Spark TPC-H Query 08 with a MapReduce randomwriter reveals the unexpected behavior. LRTrace monitors the resource metrics of a LVW container long after the application is finished. We use the memory consumption to demonstrate this situation. Figure 9 shows the related metrics and events during the lifespan of the container. The red vertical line indicates the time when the application transitions to the FINISHED state¹. In this case, container_03 keeps alive for another 14s seconds after the application is finished. Then, we check the states of container_03. The container enters the KILLING state 2s after the application is finished and stays in the state for 12s, shown by the dashed vertical blue line. Since the container is alive, it takes up about 450MB memory in the cluster. We also observe the similar behavior when running other workloads. In the worst case of a Spark Wordcount application, two containers spend more than 40 seconds in the KILLING state and occupy more than 500MB memory (250MB each), which we call zombie containers that can be only identified by LRTrace by correlating both logs and resource metrics.

The bug is caused by the fact that Yarn ResourceManager considers that the container is finished upon receiving a heartbeat from NodeManager notifying the container is in the KILLING state. This holds true in most cases since such containers are usually terminated fast enough before the application is finished. Using the current notification mechanism, another scenario is that the container is terminated before the corresponding delayed heartbeat notifying ResourceManager. The heartbeat is passively delayed due

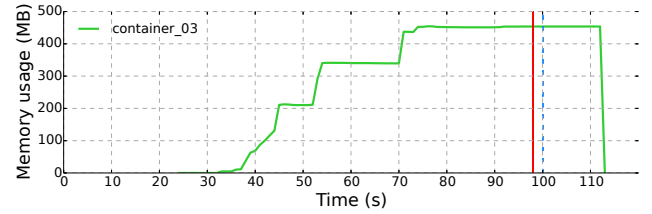


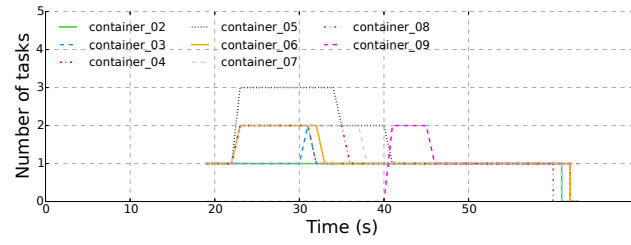
Figure 9: A container spends a long time in KILLING state. The vertical red line at the 98th second indicates that the application is finished. The vertical blue line at the 100th second is the boundary between RUNNING state and KILLING state of the container.

to high network contention. Although, ResourceManager receives the heartbeat late, resources are actually released by the container, which is also acceptable. The bug scenario occurs when the container is finished from ResourceManager perspective but is still stuck in the KILLING state. Chances are that ResourceManager allocates new containers to the same node that runs the stuck one, leading to resource contention. A possible solution is changing the notification mechanism. The NodeManager should actively send a notification to the ResourceManager after the container actually terminates. Table 5 summarizes scenarios of container termination.

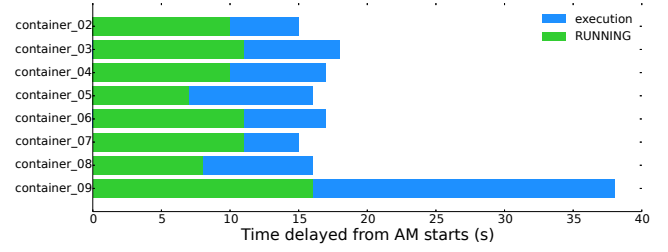
5.4 Interference Detection

Interference in cloud environments is common. It can significantly degrade the performance of applications [14]. In this experiment, we show how we find a performance anomaly by using LRTrace. We run a Spark Wordcount application on a 300MB input data set. The task assignment unbalance scenario also occurs in the application – container_09 receives no task during the first half of its execution. We also find that container_09 enters the internal

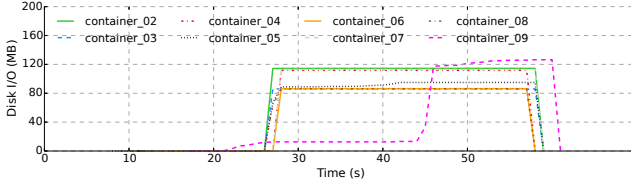
¹To avoid confusion, 'FINISHED' is the state of the application instead of the state of the container.



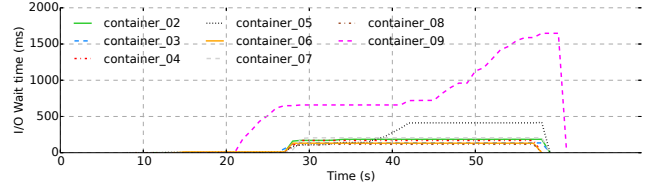
(a) number of running tasks during execution.



(b) Delays that a container enters the RUNNING state and the internal execution state.



(c) The cumulative disk I/O usage.



(d) The cumulative time spent on waiting disk service.

Figure 10: Related metrics when debugging an anomaly caused by interference in the cloud.

Slow termination	Late heartbeat	Influence
No	No	This is the normal termination.
No	Yes (passive)	Delay the scheduling for other applications. Resources are released.
Yes	No	ResourceManager is unaware of the long termination, causing resource wastage and contention.
Yes	Yes (active)	A possible solution to the bug, a heartbeat should report the container state after its actual termination.

Table 5: Summary scenarios about container termination.

execution state much later than the other containers. Figures 10(a) and 10(b) show the two scenarios, respectively. At this point, a user may consider that the unbalance is caused by the bug in the Spark scheduler (described in Section 5.2). However, after checking all the resource metrics of this application, we find the root cause is disk I/O contention on the node that runs container_09. Figures 10(c) and 10(d) show that container_09 has a longer disk I/O wait time but much lower disk I/O usage, which indicates that other processes on the same node compete for the disk throughout its execution. The contention gets more severe during 46s to 59s since the disk wait time keeps increasing drastically while there is only little disk I/O usage. The root cause of the task unbalance is that the disk I/O interference delays the start of container_09. By Figures 10(a) and 10(b), we can clearly find that container_09 receives tasks as soon as it is fully initialized.

Anomalies due to different reasons may have the same behavior. Using only logs can mislead a user to inaccurate root causes. Thus, a user may consider the root cause as a bug instead of interference if only using information from logs. LRTrace is able to identify the interference because of using both logs and resource metrics.

Summary on diagnosis The examples above show that events from logs and changes in resource consumption are closely related so that any mismatching, such as a decrease in memory without spilling, deserves further analysis. Since popular data analytics frameworks adopt a distributed parallelism model, comparing the information from different containers usually reveals the anomaly.

5.5 Feedback Control

We design and implement two plug-ins to illustrate the flexibility and effectiveness the feedback control component of LRTrace. By using Java ClassLoader class, a user-defined plug-in package can be loaded at runtime. Generally, a plug-in has about 300 lines of code but can vary due to the implementation logic.

Queue rearrangement. This plug-in is designed to rearrange the scheduling queue in the Resource Manager. The plug-in moves an application to other scheduling queue if the application meets either of the following conditions: 1) the application is pending; 2) the application is running slowly. To find a pending application is straightforward, we simply check whether its state is ACCEPTED. We define an application as a slow one if it has both of the following symptoms: 1) its memory usage is under the memory limit and it does not increase for a time threshold, and 2) it does not output log messages for a time threshold. After the plug-in finds an application that needs to be moved, it moves the application to another scheduling queue with the most available resources.

To evaluate the plug-in, we configure the scheduler with two queues: default queue and alpha queue. Each queue has half of the cluster resource. We submit three applications, Spark Wordcount, Spark KMeans and MapReduce Wordcount, to default queue, and keep one instance of each application at a time. The experiment lasts for one hour, with and without the plug-in, respectively. Figure 11 shows the number of executed applications and the execution time of applications. The plug-in improves the overall cluster throughput by 22.0% and reduce the average execution time by 18.8%.

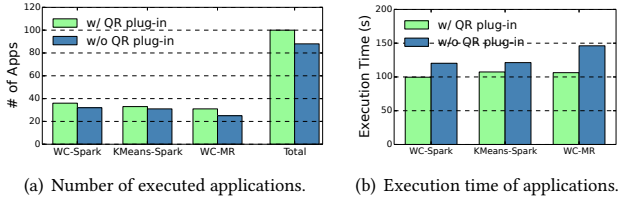


Figure 11: Evaluation of the queue rearrangement plug-in.

Application restart. This plug-in kills and restarts a stuck or failed application if the application meets user-defined conditions. During our experiments, we observe that a few applications fail at the first submission but successfully finish at the second submission even with the same configuration. We find two reasons: 1) the resource usage fluctuation in the system causes the failure, and 2) the failed application is running with underlying maintenance jobs, such as HDFS loader balancer, simultaneously.

We implement the plug-in to kill the stuck or failed applications and restart it later. The plug-in maintains a log timeout threshold for each application. If not receiving any log message of an application beyond the timeout threshold, the plug-in finds the launch command code of the application, kills the application and restarts it later. To avoid infinite killing and restarting, the plug-in maintains a maximum number of restart times for each application. If an application still cannot successfully finish after several restarting, it requires manual inspection by the users or administrators.

5.6 Performance Overhead

Before we proceed to discuss the limitations of LRTrace, we evaluate its performance by answering two questions: 1) How long does it take for a log message from its generation to being stored in the database? 2) How much performance overhead does LRTrace introduce to the targeted system?

Log arrival latency. Real-time processing is important for an on-line tracing tool since it should make the information accessible as soon as possible. We measure the latency between the time when a new message is written to the log file and the time when it is stored in the database. In this experiment, we assume that all nodes in the cluster have synchronized clocks. We write a program to generate simulated log messages that include a timestamp when the message is written (ltime). The database outputs a timestamp when storing a log message (dtime). We calculate the log arrival latency by subtracting dtime by ltime. Figure 12(a) shows that the latency generally follows a uniform distribution between 5 ms and 210 ms, which is reasonable for real-time processing.

Degradation. A good tracing tool introduces moderate performance overhead to the targeted system. We use the slowdown of an application as the measure. It is calculated by the ratio of the execution time when the applications are running with LRTrace to that without LRTrace. We run each application with and without LRTrace multiple times, and use the average execution time to calculate the slowdown. For the Spark TPC-H application, one round of execution includes running every query once. Figure 12(b) shows that the slowdown on different applications varies, with a maximum factor of 7.7%. The average slowdown is only 3.8%.

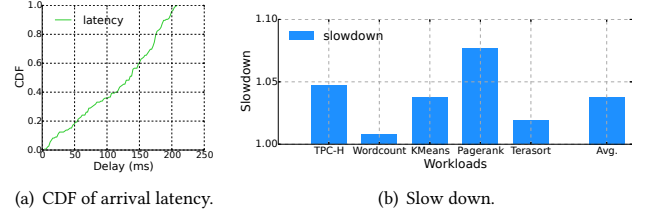


Figure 12: Performance overhead of LRTrace.

6 LIMITATIONS AND DISCUSSIONS

Limitations. LRTrace has demonstrated its efficiency in identifying new bugs and performance anomalies. It still requires users to define rules to transform raw log messages to keyed messages. Our experiments show that we can reconstruct the whole workflow of a Spark application by defining fewer than 15 rules. For a user who is not familiar with the log messages generated by the framework, it may take some time to find out log messages related to workflows and define the rules to transform logs. On the other hand, it provides flexibility for a user to add or delete rules.

LRTrace automatically extracts logs and resource metrics, leaving analysis to users. In experiments, we demonstrate that correlating log messages and resource metrics is powerful in both understanding the workflow and debugging distributed systems.

LRTrace focuses on applications that are compatible with Linux cgroup-based virtualization techniques, such as LXC and Docker. Such lightweight virtualization provides more fine-grained resource metrics compared to complex virtualization techniques such as VMs. We implement LRTrace mainly for Docker containers since it is a popular virtualization project. However, LRTrace can be extended to other container-based virtualization techniques as long as they provide APIs for monitoring.

Practical experience. We find that the top-down analysis on the data extracted by LRTrace is an effective way to identify the root cause of anomalies. When using LRTrace, users may first check the high-level events of an application and its containers, such as state transition events. Dynamic plug-in loading is an efficient way to perform feedback control at application runtime. In a cluster consisting of more than tens and hundreds of nodes manual management is, if not impossible, hard to be practical. By using the developed feedback control component, users can be relieved from the burden of conducting manual management tasks.

7 RELATED WORK

There are extensive studies on tracing and troubleshooting. They can be generally divided into two categories: 1) intrusive approach, and 2) log analysis.

Intrusive approach. Quite a few tracing tools [12, 15, 16, 24, 25] adopt this approach to obtain workflow information from inside the targeted systems. The key advantage of intrusive approaches is that extracted information can be unambiguously identified, which increases the accuracy of workflow reconstruction. Since a user request usually invokes services from different layers, tools like Magpie [12] further apply this approach to cross-layer systems. This provides an overview of the user request, which facilitates the

debugging process. However, some intrusive tools bring considerable overhead since they always collect all the defined information as long as the targeted systems are running.

To solve this problem, DTrace [15] firstly proposed dynamic instrumentation. Dynamic instrumentation leverages techniques like Aspect Oriented Programming to dynamically extract the intended information. A representative tool in this category is Pivot Tracing [24]. It requires minor modification to the source code of targeted systems and dynamically installs trace points to collect only the information intended by users. It brings almost no overhead when turned off. In contrast to the intrusive approach, LRTrace is a non-intrusive profiling tool so that it requires no modification to the source code of the targeted system, which makes it more easily to be deployed.

Log analysis. Logs output by systems contain rich information, such as lifespans of objects, events in the system and so on [17]. Studies [4, 33] focus on how to identify fields in a log messages. Effective log analysis is a powerful method for troubleshooting. A number of studies focus on applying machine learning methods to log analysis [13, 19, 26, 28, 29]. The main goal of this method is automatically identifying potential anomalies, which narrows down the range for manual debugging. Algorithms are trained by logs generated by normal workflows. Once done, it is able to distinguish logs generated by anomalies. For example, PerfScope [19] applies a top-down hierarchical clustering algorithm on system logs to group those execution units that share similar behaviors. Then, it uses the nearest neighbor algorithm to identify the outliers. The accuracy of machine learning methods depends on two factors: 1) the amount of training data, and 2) the parameter configuration. Therefore, it is suitable for repeated long running applications. However, such methods are unlikely to identify the root causes of anomalies.

Another category is static analysis, such as Sherlog [30] and Iprof [32]. This method performs source code analysis to identify the corresponding logging locations. Iprof [32] is a representative. Static analysis provides more accuracy as well as less overhead when reconstructing the workflow since it is aware of the corresponding locations of logging statements in the source code but non-intrusive. However, It needs to perform static analysis every time the targeted systems are upgraded, which is similar to intrusive approaches. It relies on the bytecode, a feature adopted by languages using JVM such as Java and Scala. Systems written by C/C++ needs further processing to support static analysis. LRTrace does not need to perform static analysis, which makes it suitable for more systems.

8 CONCLUSION AND FUTURE WORK

In this paper, we present LRTrace, a non-intrusive tracing tool that extracts information about both logs and resource metrics at runtime. LRTrace transfers a raw log message to a uniform data structure called keyed message. By using keyed messages, users can easily reconstruct the workflow including the events and the amount of processed data if recorded. Lightweight virtualization techniques provide the opportunity to access fine-grained per container resource metrics. By correlating these two kinds of information, a user can efficiently find the anomaly and locate its root cause. Experiments show that LRTrace helps users in understanding

workflows, finding anomalies caused by either bugs or interference while incurring low overhead.

In the future, we plan to use machine learning methods or rule-based methods to automatically build the relationship between logs and resource metrics, which further takes the burdens off users.

9 ACKNOWLEDGEMENTS

This research was supported in part by U.S. NSF award CNS-1422119.

REFERENCES

- [1] Docker. <https://linuxcontainers.org/lxc/>.
- [2] Graphite. <https://graphite.readthedocs.io/>.
- [3] LXC. <https://linuxcontainers.org/lxc/>.
- [4] Logstash. <https://www.elastic.co/products/logstash/>.
- [5] OpenTSDB. <http://opentsdb.net/>.
- [6] Spark-19371. <https://issues.apache.org/jira/browse/SPARK-19371/>.
- [7] TPC-H. <http://www.tpc.org/tpch/>.
- [8] YARN-6976. <https://issues.apache.org/jira/browse/YARN-6976/>.
- [9] Apache kafka. <https://kafka.apache.org/>.
- [10] Apache Log4j 2. <https://logging.apache.org/log4j/2.x/>.
- [11] SLF4J. <https://www.slf4j.org/>.
- [12] P. Barham, A. Donnelly, R. Issacs, and R. Mortier. Using magpie for request extraction and workload modelling. In *Proc. of USENIX OSDI*, 2004.
- [13] I. Beschastnikh, Y. Brun, S. Schneider, M. Sloan, and M. D. Ernst. Leveraging existing instrumentation to automatically infer invariant-constrained models. In *Proc. of ACM SIGSOFT ESEC/FSE*, 2011.
- [14] X. Bu, J. Rao, and C.-z. Xu. Interference and locality-aware task scheduling for MapReduce applications in virtual clusters. In *Proc. of ACM HPDC*, 2013.
- [15] B. M. Cantrill, M. W. Shapiro, and A. H. Leventhal. Dynamic instrumentation of production systems. In *Proc. of USENIX ATC*, 2004.
- [16] W. Chen, J. Rao, and X. Zhou. Preemptive, low latency datacenter scheduling via lightweight virtualization. In *Proc. of USENIX ATC*, 2017.
- [17] W. Chen, A. Pi, S. Wang, and X. Zhou. Characterizing scheduling delay for low-latency data analytic workloads. In *Proc. of IEEE IPDPS*, 2018.
- [18] Z. Chothia, J. Liagouris, D. Dimitrova, and T. Roscoe. Online reconstruction of structural information from datacenter logs. In *Proc. of ACM Eurosys*, 2017.
- [19] D. J. Dean, H. Nguyen, X. Gu, H. Zhang, J. Rhee, Nipun, Arora, and G. Jiang. Perfscope: Practical online server performance bug inference in production cloud computing infrastructures. In *Proc. of ACM SoCC*, 2014.
- [20] J. Dean and S. Ghemawat. MapReduce: simplified data processing on large clusters. In *Proc. of ACM Communications*, 2008.
- [21] B. Hindman, A. Konwinski, M. Zaharia, A. Ghodsi, A. D. Joseph, R. H. Katz, S. Shenker, and I. Stoica. Mesos: A platform for fine-grained resource sharing in the data center. In *Proc. of USENIX NSDI*, 2011.
- [22] S. Huang, J. Huang, J. Dai, T. Xie, and B. Huang. The HiBench benchmark suite: Characterization of the mapreduce-based data analysis. In *Proc. of IEEE Data Engineering Workshops (ICDEW)*, 2010.
- [23] A. Kivity, Y. Kamay, D. Laor, U. Lublin, and A. Liguori. kvm: the linux virtual machine monitor. In *Proc. of Linux symposium*, 2007.
- [24] J. Mace, R. Roelke, and R. Fonseca. Pivot tracing: Dynamic causal monitoring for distributed systems. In *Proc. of ACM SOSP*, 2015.
- [25] M. Mejbah ul Alam, T. Liu, G. Zeng, and A. Muzahid. Syncperf: Categorizing, detecting, and diagnosing synchronization performance bugs. In *Proc. of ACM Eurosys*, 2017.
- [26] K. Nagaraj, C. Killian, and J. Neville. Structured comparative analysis of systems logs to diagnose performance problems. In *Proc. of USENIX NSDI*, 2012.
- [27] V. K. Vavilapalli, A. C. Murthy, C. Douglas, S. Agarwal, M. Konar, R. Evans, T. Graves, J. Lowe, H. Shah, S. Seth, et al. Apache Hadoop YARN: Yet another resource negotiator. In *Proc. of ACM SoCC*, 2013.
- [28] W. Xu, L. Huang, A. Fox, D. Patterson, and M. I. Jordan. Detecting large-scale system problems by mining console logs. In *Proc. of ACM SOSP*, 2009.
- [29] X. Yu, P. Joshi, J. Xu, and G. Jin. CloudSeer: Workflow monitoring of cloud infrastructures via interleaved logs. In *Proc. of ACM ASPLOS*, 2016.
- [30] D. Yuan, H. Mai, W. Xiong, L. Tan, Y. Zhou, and S. Pasupathy. Sherlog: error diagnosis by connecting clues from run-time logs. In *Proc. of ACM ASPLOS*, 2010.
- [31] M. Zaharia, M. Chowdhury, M. J. Franklin, S. Shenker, and I. Stoica. Spark: Cluster computing with working sets. In *Proc. of USENIX HOTCLOUD*, 2010.
- [32] X. Zhao, Y. Zhang, D. Lion, M. FaizanUllah, Y. Luo, D. Yuan, and M. Stumm. Iprof: A non-intrusive request flow profiler for distributed systems. In *Proc. of USENIX OSDI*, 2014.
- [33] X. Zhao, K. Rodrigues, Y. Luo, D. Yuan, and M. Stumm. Non-intrusive performance profiling for entire software stacks based on the flow reconstruction principle. In *Proc. of USENIX OSDI*, 2016.